

Rapport de projet informatique

AI-Sentinel : Système de détection d'intrusions

Projet réalisé du 26 mars 2026 au 23 mai 2026

Dépôt GitHub : <https://github.com/aakkashthirulo-spec/AI-Sentinel>

Membres du groupe

Thirulokkumar Aakkash - 45018080

Cornu Valentin - 45016459

Demian Nicolae - 45014894

Remerciements

Nous tenons à remercier l'équipe pédagogique pour ce projet qui nous a permis d'allier l'intelligence artificielle aux problématiques concrètes de la cybersécurité.

Table des matières

1	Introduction	4
2	Environnement de travail	4
3	Description du projet et objectifs	4
3.1	Analyse de flux en temps réel	4
3.2	Fiabilité et intégrité	5
4	Bibliothèques, outils et technologies	5
5	Travail réalisé	5
5.1	Architecture du Projet “AI-Sentinel” : Approche Modulaire et Multilangage	5
5.1.1	Le Capteur Réseau : Sentinel-Sensor (Code en C)	5
5.1.2	Le Moteur de Détection IA : Sentinel-AI (Code en Python) . . .	6
6	Difficultés rencontrées	7
7	Bilan	7
7.1	Conclusion	7
7.2	Perspectives	8
8	Bibliographie	9
9	Webographie	10
A	Cahier des charges	11
B	Exemple d’exécution du projet	11
C	Manuel utilisateur	11

1 Introduction

AI-Sentinel est une solution de surveillance conçue pour sécuriser les infrastructures critiques. Le système analyse les métadonnées de connexion afin d'identifier en temps réel des comportements suspects. L'enjeu est de garantir la disponibilité des services face à des menaces de plus en plus sophistiquées.

2 Environnement de travail

Le développement a été réalisé de manière collaborative avec **GitHub** pour la gestion du code source et **Overleaf** pour la rédaction de ce rapport. Le système a été testé dans un environnement **Linux**, ce qui facilite la manipulation des journaux système.

3 Description du projet et objectifs

L'objectif du projet est de fournir une réponse rapide face aux cyberattaques. Notre architecture repose sur une validation stricte des données échangées entre les différents modules.



FIGURE 1 – Tux, le pingouin (environnement de test Linux)

Le tableau 1 présente les types d'anomalies que nous avons choisi de traiter.

Type d'anomalie de trafic	Niveau de criticité	Responsable du module
Injection / Paquet corrompu (C)	Élevé	Nicolae
Déviance comportementale (Python)	Moyen	Valentin
Violation d'intégrité de flux	Critique	Aakkash

TABLE 1 – Types de détections et gestion multi-niveaux du système

3.1 Analyse de flux en temps réel

Le système doit être capable de traiter des flux de données continus[cite : 28]. Le module en C assure un filtrage initial pour ne transmettre que les données formatées correctement[cite : 29].

3.2 Fiabilité et intégrité

L’objectif secondaire est d’empêcher toute corruption du moteur d’IA. Si le flux de données est interrompu ou modifié, le système doit se mettre en sécurité[cite : 31].

4 Bibliothèques, outils et technologies

Nous avons utilisé le compilateur **GCC** pour optimiser les performances du module en C. Pour la partie Python, nous avons exploité des bibliothèques de calcul statistique pour analyser les écarts volumétriques et de fréquences du trafic. Les outils d’IA générative ont été intégrés comme assistants de programmation pour valider l’allocation mémoire en C et structurer le pipeline de données.

5 Travail réalisé

- **Réalisé** : Module d’interception et d’écriture de fichier de trafic CSV en C, intégration de filtres de couleurs dans le terminal Python pour distinguer visuellement les risques (faible, moyen, critique), algorithme IA d’analyse comportementale et script de contrôle d’intégrité du flux.
- **Non réalisé** : La coupure automatique de l’interface réseau (prévention active) n’a pas pu être intégrée, faute de droits administrateurs étendus sur notre environnement de test.
- **Répartition** : Nicolae s’est chargé du module de capture en C (performance) et il s’est occupé du fichier `sentinel_main.py` qui permet de lancer et d’arrêter simultanément, via une interface graphique simple, le module de capture de trafic en C et le moteur d’analyse comportementale en Python, Valentin du moteur d’IA en Python (analyse), et Aakash de la coordination globale, de l’intégration du pipeline et de la sécurité du flux de confiance.

5.1 Architecture du Projet “AI-Sentinel” : Approche Modulaire et Multilangage

Pour répondre aux exigences de performance et d’accessibilité de notre système de détection d’intrusions (IDS), nous avons opté pour une architecture strictement modulaire. Le projet est divisé en trois programmes distincts qui communiquent entre eux. Cette séparation des tâches permet d’utiliser les forces spécifiques de chaque langage (C et Python) tout en rendant le code plus facile à maintenir.

5.1.1 Le Capteur Réseau : Sentinel-Sensor (Code en C)

- **Rôle** : C’est le “bouclier” et les “yeux” de notre programme. Il s’occupe de la capture des données brutes du réseau Wi-Fi.
- **Pourquoi le langage C ?** Le C a été imposé et choisi pour sa proximité avec le matériel (bas niveau) et sa gestion stricte de la mémoire. Pour écouter un réseau en temps réel, il faut un langage extrêmement rapide qui ne sature pas le processeur.
- **Fonctionnement concret** : Ce programme intercepte les trames réseaux (IP source, ports, taille du payload, contenu) et les formate instantanément sous

forme de texte dans un fichier de journalisation continu (le fichier `traffic_logs.csv`). Il agit comme un producteur de données.

5.1.2 Le Moteur de Détection IA : Sentinel-AI (Code en Python)

- **Rôle** : C’est le “cerveau” du projet. Il dispose d’une logique algorithmique pour détecter les anomalies et les attaques.
- **Pourquoi le langage Python ?** Python excelle dans le traitement de données et l’implémentation de logique algorithmique complexe. Il est idéal pour calculer des statistiques en temps réel (comme les Z-scores sur la taille des paquets) et manipuler des chaînes de caractères pour détecter des signatures d’attaques.
- **Fonctionnement concret** : Ce script lit le fichier CSV généré par le programme C en “Live Mode”. Il applique des filtres de sécurité pour identifier les comportements malveillants :
 - **Flood / Scan de ports** : Détectés par une fréquence anormale de requêtes.
 - **Brute Force / Injections (ex : SQL)** : Détectés par l’analyse du contenu du payload.
- **Score de risque** : Le programme calcule un score final et affiche une interface console dynamique et colorée alertant l’utilisateur sur la gravité de la menace.

Pour répondre aux exigences de performance et d’accessibilité de notre système de détection d’intrusions (IDS), nous avons opté pour une architecture strictement modulaire. Le projet est divisé en trois programmes distincts qui communiquent entre eux. Cette séparation des tâches permet d’utiliser les forces spécifiques de chaque langage (C et Python) tout en rendant le code plus facile à maintenir.

Le Capteur Réseau : Sentinel-Sensor (Code en C)

- **Rôle** : C’est le “bouclier” et les “yeux” de notre programme. Il s’occupe de la capture des données brutes du réseau Wi-Fi.
- **Pourquoi le langage C ?** Le C a été imposé et choisi pour sa proximité avec le matériel (bas niveau) et sa gestion stricte de la mémoire. Pour écouter un réseau en temps réel, il faut un langage extrêmement rapide qui ne sature pas le processeur.
- **Fonctionnement concret** : Ce programme intercepte les trames réseaux (IP source, ports, taille du payload, contenu) et les formate instantanément sous forme de texte dans un fichier de journalisation continu (le fichier `traffic_logs.csv`). Il agit comme un producteur de données.

Le Moteur de Détection IA : Sentinel-AI (Code en Python)

- **Rôle** : C’est le “cerveau” du projet. Il dispose d’une logique algorithmique pour détecter les anomalies et les attaques.
- **Pourquoi le langage Python ?** Python excelle dans le traitement de données et l’implémentation de logique algorithmique complexe. Il est idéal pour calculer des statistiques en temps réel (comme les Z-scores sur la taille des paquets) et manipuler des chaînes de caractères pour détecter des signatures d’attaques.
- **Fonctionnement concret** : Ce script lit le fichier CSV généré par le programme C en “Live Mode”. Il applique des filtres de sécurité pour identifier les comportements malveillants :
 - **Flood / Scan de ports** : Détectés par une fréquence anormale de requêtes.

- **Brute Force / Injections (ex : SQL)** : Détectés par l’analyse du contenu du payload.
- **Score de risque** : Le programme calcule un score final et affiche une interface console dynamique et colorée alertant l’utilisateur sur la gravité de la menace.

L’Interface Utilisateur : Sentinel-Launcher (Orchestrateur Python)

- **Rôle** : Rendre l’outil accessible à un utilisateur novice (“lambda”) via une interface graphique simple (bouton Activer/Désactiver).
- **Le “Pourquoi” de cette approche** : En cybersécurité, un outil n’est efficace que s’il est utilisé. Demander à un utilisateur sans compétences informatiques d’ouvrir un terminal, de compiler du C, puis d’exécuter un script Python avec des arguments est inenvisageable. Nous avons donc décidé de masquer toute la complexité technique derrière une application “Plug & Play”. L’utilisateur clique sur un bouton, et le système se protège tout seul.
- **Le “Comment” (Implémentation technique)** : Nous avons créé un script Python supplémentaire qui agit comme un chef d’orchestre.
 - **L’interface visuelle** : Elle est générée grâce à la bibliothèque native `tkinter`, qui permet de dessiner une fenêtre applicative (boutons, labels d’état) sans avoir besoin d’installer de modules externes complexes.
 - **L’orchestration (Le module subprocess)** : Lorsque l’utilisateur clique sur “Activer”, ce programme utilise la bibliothèque `subprocess`. Cette bibliothèque permet de lancer d’autres exécutables en arrière-plan (processus enfants). L’orchestrateur va donc lancer silencieusement notre Sentinel-Sensor (C) en tâche de fond, puis ouvrir la console de notre Sentinel-AI (Python). Ainsi, les deux programmes s’exécutent simultanément et communiquent de manière totalement transparente pour l’utilisateur.

postseason

6 Difficultés rencontrées

La principale difficulté a été d’assurer une communication fiable entre le module en C et le programme Python. Il a fallu définir un format de données clair afin d’éviter les erreurs lors du traitement des fichiers CSV. Nous avons aussi dû gérer les lignes incomplètes, les caractères inattendus et les journaux suspects. Les tests ont demandé du temps, car plusieurs cas devaient être simulés pour vérifier la fiabilité du système. Ces difficultés nous ont permis de mieux comprendre l’importance d’une architecture simple, robuste et sécurisée.

7 Bilan

7.1 Conclusion

Le projet atteint ses objectifs en proposant un système capable d’analyser des journaux et de repérer des anomalies. L’utilisation du C permet un filtrage rapide, tandis que Python facilite l’analyse et la détection intelligente. Cette organisation rend le projet plus clair, plus fiable et plus simple à faire évoluer. Même si certaines améliorations restent possibles, comme l’ajout d’alertes automatiques, la base du système

est fonctionnelle. Ce travail nous a permis de mettre en pratique nos compétences en programmation et en cybersécurité.

7.2 Perspectives

Une amélioration possible serait l'ajout d'une interface graphique pour visualiser les alertes en temps réel sur un tableau de bord.

8 Bibliographie

Références

- [GEM26] Google AI, *Assistant de génération de code, d’audit de sécurité et de mise en page L^AT_EX Gemini*, version 2026.
- [MAM26] MIA, *Assistant d’analyse logique et de co-conception algorithmique Python Mammouth AI*, version 2026.

9 Webographie

Références

[GH] Dépôt public du projet : <https://github.com/aakkashthirulo-spec/AI-Sentinel>

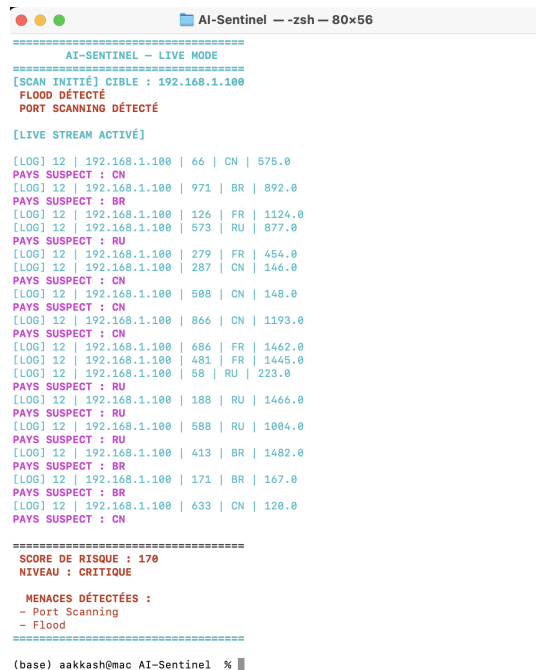
Annexe A : Cahier des charges

Le système doit auditer en continu le trafic réseau afin d'isoler les comportements malveillants. Le programme en C réalise un contrôle de formatage amont, puis le script Python interprète les résultats statistiques. En cas de flux corrompu ou d'altération du fichier intermédiaire, le programme doit immédiatement lever une alerte critique et se mettre en sécurité.

Annexe B : Exemple d'exécution du projet

```
Last login: Fri May 22 11:21:57 on ttys002
/var/folders/3w/5_cg27xd3ts31_3hl9g7l0kw0000gn/T/geany_run_script_GY5RP3.sh ; ex
it;
(base) aakkash@MacBook-Air-de-TAakkash ~ % /var/folders/3w/5_cg27xd3ts31_3hl9g7l
0kw0000gn/T/geany_run_script_GY5RP3.sh ; exit;
[C-SENTINEL] Début de la capture sur l'interface (Simulation)...
[C-SENTINEL] Capture terminée. Logs générés avec succès.
```

FIGURE 2 – Capture d'écran du terminal lors de la capture des flux par le module C.



```
AI-Sentinel -- zsh -- 80x56
=====
AI-SENTINEL ~ LIVE MODE
=====
[SCAN INITIÉ] CIBLE : 192.168.1.100
FLOOD DÉTECTÉ
PORT SCANNING DÉTECTÉ

[LIVE STREAM ACTIVÉ]

[LOG] 12 | 192.168.1.100 | 66 | CN | 575.0
PAYS SUSPECT : CN
[LOG] 12 | 192.168.1.100 | 971 | BR | 892.0
PAYS SUSPECT : BR
[LOG] 12 | 192.168.1.100 | 126 | FR | 1124.0
[LOG] 12 | 192.168.1.100 | 573 | RU | 877.0
PAYS SUSPECT : RU
[LOG] 12 | 192.168.1.100 | 279 | FR | 454.0
[LOG] 12 | 192.168.1.100 | 287 | CN | 146.0
PAYS SUSPECT : CN
[LOG] 12 | 192.168.1.100 | 508 | CN | 148.0
PAYS SUSPECT : CN
[LOG] 12 | 192.168.1.100 | 866 | CN | 1193.0
PAYS SUSPECT : CN
[LOG] 12 | 192.168.1.100 | 686 | FR | 1462.0
[LOG] 12 | 192.168.1.100 | 481 | FR | 1445.0
[LOG] 12 | 192.168.1.100 | 58 | RU | 223.0
PAYS SUSPECT : RU
[LOG] 12 | 192.168.1.100 | 188 | RU | 1466.0
PAYS SUSPECT : RU
[LOG] 12 | 192.168.1.100 | 588 | RU | 1004.0
PAYS SUSPECT : RU
[LOG] 12 | 192.168.1.100 | 413 | BR | 1482.0
PAYS SUSPECT : BR
[LOG] 12 | 192.168.1.100 | 171 | BR | 167.0
PAYS SUSPECT : BR
[LOG] 12 | 192.168.1.100 | 633 | CN | 120.0
PAYS SUSPECT : CN

=====
SCORE DE RISQUE : 170
NIVEAU : CRITIQUE

MENACES DÉTECTÉES :
- Port Scanning
- Flood
=====

(base) aakkash@mac AI-Sentinel %
```

FIGURE 3 – Détection d'une anomalie volumétrique par le moteur IA en Python.

Annexe C : Manuel utilisateur

Pour exécuter le système complet de surveillance, ouvrez votre terminal Linux et lancez la commande suivante :

```
python3 sentinel_main.py trafic_reseau.csv
```